

Piglet with Block Errors and Augmented Gabidulin Codes (BAG-Piglet)

1	Description of BAG-Piglet	2
1.1	Overview of BAG-Piglet	2
1.2	Notation and Some Concepts	2
1.3	IND-CPA Secure BAG-Piglet.PKE	5
1.4	IND-CCA2 Secure BAG-Piglet.KEM	6
1.5	Parameter Selection	6
2	Design Rationale	8
2.1	Difficult Problems for Code-based Public-key Cryptography	8
2.2	Criterion for Parameter Choices	9
2.3	Augmented Gabidulin Codes and Decoding	10
2.4	Statement of Design	11
3	Security Analysis	12
3.1	Theoretical Security Proof	12
3.2	Security Analysis on Known Attacks	14
3.2.1	O-J Combinatorial Attacks	15
3.2.2	PRR Combinatorial Attacks	16
3.2.3	Algebraic Attacks	18
3.3	Estimates on Concrete Security Strength	20
4	Performance Analysis	20
5	The Features of BAG-Piglet	21
5.1	The features of BAG-Piglet	21
5.2	Comparison with other schemes	23
	References	23

1 Description of BAG-Piglet

1.1 Overview of BAG-Piglet

We first present an IND-CPA secure public-key encryption scheme BAG-Piglet.PKE. By applying the salted variant of the Fujisaki–Okamoto (SFO) transform, we provide an IND-CCA2 secure KEM, denoted as BAG-Piglet.KEM. We provide concrete parameter sets targeting classical security levels of 128, 256, 384, and 512 bits, while achieving 80, 128, 192, and 256 bits of quantum security, respectively. In addition, we present a comprehensive security analysis that covers combinatorial attacks and algebraic attacks in the presence of blended noise, along with an implementation of the scheme.

1.2 Notation and Some Concepts

We introduce the necessary notations and concepts used throughout the construction of BAG-Piglet.PKE and BAG-Piglet.KEM.

Basic algebraic structures. Let q be a prime power and let $\mathbb{F}_{q^m}$ denote the finite extension field of degree m over $\mathbb{F}_q$. For BAG-Piglet, we take $q = 2$. Let $P(X) \in \mathbb{F}_q[X]$ be an irreducible polynomial of degree n , and define the quotient ring

$$\mathcal{R} := \mathbb{F}_{q^m}[X]/(P(X)).$$

All algebraic operations are performed over $\mathbb{F}_{q^m}$ or $\mathcal{R}$ as specified. Vectors are denoted by bold lowercase letters, and matrices by uppercase letters. Unless otherwise specified, all vectors are row vectors.

Support and rank weight. For any $\mathbf{V} = (v_{i,j}) \in \mathcal{R}^{r \times t}$, the support of $\mathbf{V}$ is defined as the $\mathbb{F}_q$ -subspace generated by all its entries, i.e., $\text{Supp}(\mathbf{V}) = \langle v_{i,j} : 1 \leq i \leq r, 1 \leq j \leq t \rangle_{\mathbb{F}_q}$. The rank weight of $\mathbf{V}$ is defined as $w_R(\mathbf{V}) = \dim(\text{Supp}(\mathbf{V}))$, sometimes also denoted as $w_{\mathbf{V}}$.

Linear code. An $[n, k]_{q^m}$ -linear code $\mathcal{C}$ is a k -dimensional subspace of $\mathbb{F}_{q^m}^n$. It can be equivalently represented by a full-rank generator matrix $\mathbf{G} \in \mathbb{F}_{q^m}^{k \times n}$ or a full-rank parity-check matrix $\mathbf{H} \in \mathbb{F}_{q^m}^{(n-k) \times n}$, such that $\mathcal{C} = \{\mathbf{xG} : \mathbf{x} \in \mathbb{F}_{q^m}^k\}$ or $\mathcal{C} = \{\mathbf{x} \in \mathbb{F}_{q^m}^n : \mathbf{Hx}^\top = \mathbf{0}\}$.

Syndrome. The vector $\mathbf{Hv}^\top$ is called the *syndrome* of $\mathbf{v}$ with respect to $\mathbf{H}$.

Minimum distance. The minimum rank distance of a linear code $\mathcal{C}$, denoted by $d_R(\mathcal{C})$, is the minimum rank weight among all non-zero codewords in $\mathcal{C}$. Such a code can uniquely decode up to $\left\lfloor \frac{d_R(\mathcal{C})-1}{2} \right\rfloor$ errors, and is typically denoted by $[n, k, d_R(\mathcal{C})]$.

The product of two vectors. Let n be a positive integer and let $P \in \mathbb{F}_q[X]$ be a polynomial of degree n . For any vectors $\mathbf{u}, \mathbf{v} \in \mathbb{F}_{q^m}^n$, we define their product as $\mathbf{u} \cdot \mathbf{v} \stackrel{\text{def}}{=} U(X)V(X) \bmod P$, where $U(X) = \sum_{i=0}^{n-1} x_i X^i$ and $V(X) = \sum_{i=0}^{n-1} y_i X^i$.

Ideal matrix. Let $P \in \mathbb{F}_q[X]$ be a polynomial of degree n , and let $\mathbf{v} \in \mathbb{F}_{q^m}^n$. The ideal matrix generated by $\mathbf{v}$ is defined as

$$\mathcal{IM}_P(\mathbf{v}) \stackrel{\text{def}}{=} \begin{pmatrix} \mathbf{v}(X) \bmod P \\ X\mathbf{v}(X) \bmod P \\ \vdots \\ X^{n-1}\mathbf{v}(X) \bmod P \end{pmatrix} \in \mathbb{F}_{q^m}^{n \times n}.$$

When P is clear from the context, we simply write $\mathcal{IM}(\mathbf{v})$.

Isomorphism. The polynomial ring $\mathcal{R}$ is isomorphic to $\mathbb{F}_{q^m}^n$ as an $\mathbb{F}_{q^m}$ -algebra via the mapping $\pi : \sum_{i=0}^{n-1} v_{i+1} X^i \mapsto (v_1, \dots, v_n)$.

Ideal codes. Let $P(X) \in \mathbb{F}_q[X]$ be a polynomial of degree n . An $[\ell n, sn]$ linear code $\mathcal{C}$ over $\mathbb{F}_{q^m}$ is called an (ℓ, s) -ideal code if it admits a generator matrix of the form

$$\mathbf{G} = \left(\begin{array}{c|ccc} \mathbf{I}_{sn} & \mathcal{IM}(\mathbf{g}_{1,1}) & \cdots & \mathcal{IM}(\mathbf{g}_{1,\ell-s}) \\ & \vdots & \ddots & \vdots \\ & \mathcal{IM}(\mathbf{g}_{s,1}) & \cdots & \mathcal{IM}(\mathbf{g}_{s,\ell-s}) \end{array} \right),$$

where $\mathbf{g}_{i,j} \in \mathbb{F}_{q^m}^n$ for $1 \leq i \leq s$, $1 \leq j \leq \ell - s$.

Augmented Gabidulin code. Let $(nn_1, n', k, m) \in \mathbb{N}^4$ satisfy $k \leq n' \leq m \leq n$. Let $\mathbf{g} = (g_1, \dots, g_{n'}) \in \mathbb{F}_{q^m}^{n'}$ satisfy $w_R(\mathbf{g}) = n'$, and let $\bar{\mathbf{g}} \stackrel{\text{def}}{=} (\mathbf{g} \mid \mathbf{0}_{nn_1-n'}) \in \mathbb{F}_{q^m}^n$. The augmented Gabidulin code $\mathcal{G}_{\bar{\mathbf{g}}}^+(nn_1, n', k, m)$ is the $[nn_1, k]_{q^m}$ code defined by

$$\mathcal{G}_{\bar{\mathbf{g}}}^+(nn_1, n', k, m) \stackrel{\text{def}}{=} \{P(\bar{\mathbf{g}}), \deg_q(P) < k\},$$

where

$$P(\bar{\mathbf{g}}) \stackrel{\text{def}}{=} (P(g_1), \dots, P(g_{n'}), \mathbf{0}_{nn_1-n'}),$$

and P is a q -linearized polynomial. Its decoding algorithm is denoted by $\mathcal{D}_{\mathcal{G}_{\bar{\mathbf{g}}}^+}(\cdot)$, which corrects rank errors up to the designed decoding radius, which corrects up to

$$\delta \stackrel{\text{def}}{=} \left\lfloor \frac{n' - k + \varepsilon}{2} \right\rfloor,$$

with decryption failure rate

$$\text{DFR}(nn_1, n', \delta, \varepsilon) = q^{\delta(n'-nn_1)} \sum_{i=0}^{\varepsilon-1} \prod_{j=0}^{i-1} \frac{(q^\delta - q^j)(q^{(nn_1-n')} - q^j)}{q^i - q^j}.$$

where

$$\varepsilon \in \{1, 2, \dots, \min(nn_1 - n', n' - k)\}.$$

Fold and Unfold. For a vector $\mathbf{v} \in \mathbb{F}_{q^m}^{nn_1}$, the folding operator

$$\text{Fold}(\mathbf{v}) \in \mathcal{R}^{n_1}$$

maps $\mathbf{v}$ into a ring representation, and **Unfold** denotes its inverse.

Structure of error vectors. For a positive integer ℓ , $\mathbf{n} = (n, \dots, n) \in \mathbb{Z}^\ell$, and an ℓ -block error vector $\mathbf{e} = (\mathbf{e}_1, \dots, \mathbf{e}_\ell)$, $\mathbf{e}_i \in \mathbb{F}_{q^m}^n$, define the ℓ -block error if $\text{Supp}(\mathbf{e}_i) \cap \text{Supp}(\mathbf{e}_j) = \{0\}$, for any $i \neq j, 1 \leq i \leq \ell, 1 \leq j \leq \ell$.

For a given rank weight vector $\mathbf{t} = (t_1, \dots, t_\ell) \in \mathbb{Z}^\ell$, and $\mathbf{n} = (n, \dots, n) \in \mathbb{Z}^\ell$, define

$$\mathcal{S}_{\mathbf{a}}^{\ell n}(\mathbb{F}_{q^m}) = \{\mathbf{e} = (\mathbf{e}_1, \dots, \mathbf{e}_\ell) \in \mathbb{F}_{q^m}^{\ell n} \mid \mathbf{e}_i \in \mathbb{F}_{q^m}^n, \text{wt}_R(\mathbf{e}_i) = t_i\}.$$

Random oracles and hash functions. We use `shake256`($\cdot$) as a pseudorandom generator that expands a seed into structured algebraic objects. In the KEM transformation, we further use two hash functions

$$\mathcal{G} : \{0, 1\}^* \rightarrow \{0, 1\}^{512}, \quad \mathcal{K} : \{0, 1\}^* \rightarrow \{0, 1\}^{512},$$

instantiated by SHA3-512 and SHAKE256, respectively. The function $\mathcal{ID}(\cdot)$ denotes a deterministic extraction function that derives entropy from the public key.

Key generation seeds. The key generation algorithm uses two independent seeds

$$\rho, \sigma \xleftarrow{\$} \{0, 1\}^\lambda,$$

where λ is the security parameter, the seed1 ρ is used to randomly generate the public matrix $\mathbf{h} \in \mathcal{R}^2$ and $\mathbf{g}$ in the Augmented Gabidulin code. The seed2 σ is used to generate the secret key $\mathbf{x} \in \mathcal{R}$ and $\mathbf{y} \in \mathcal{R}^2$ by sampling uniformly from $\mathcal{S}_{\mathbf{a}}^{3n}(\mathbb{F}_{q^m})$ for a given small-rank vector $\mathbf{a}$.

Encryption seed. During encryption, the seed3 θ is used to generate $\mathbf{r}_1 \in \mathcal{R}^{n_1}, \mathbf{e} \in \mathcal{R}^{n_1}, \mathbf{R}_2 \in \mathcal{R}^{2n_1}$ by sampling uniformly from the finite set $\mathcal{S}_{\mathbf{b}}^{4nn_1}(\mathbb{F}_{q^m})$ for a given small-rank weight vector b .

Message and ciphertext spaces. Messages are vectors $\mathbf{m} \in \mathbb{F}_{q^m}^k$. Ciphertexts are pairs $\mathbf{C} = (\mathbf{u}, \mathbf{v})$ with $\mathbf{u} \in \mathcal{R}^{2n_1}$ and $\mathbf{v} \in \mathcal{R}^{n_1}$. All objects are encoded into byte strings using compact representations.

KEM-specific notation. In the salted variant of the Fujisaki–Okamoto transform, we use randomness

$$(k, \theta) \leftarrow \mathcal{G}(\mathcal{ID}(\text{pk}), \mathbf{m}, \text{salt}),$$

and derive the session key as

$$K := \mathcal{K}(k, \text{ct}).$$

The decapsulation algorithm uses implicit rejection with a fallback value $\xi \in \{0, 1\}^{512}$.

1.3 IND-CPA Secure BAG-Piglet.PKE

In this section, we first present an IND-CPA secure public-key encryption scheme, BAG-Piglet.PKE, as shown in Algorithm 1.

BAG-Piglet.PKE :

KeyGen(1^λ):

- Sample seeds $\rho, \sigma \xleftarrow{\$} \{0, 1\}^\lambda$.
- Sample uniformly $\mathbf{h} = (\mathbf{h}_1, \mathbf{h}_2) \in \mathcal{R}^2 := \text{shake256}(\rho)$.
- $(\mathbf{x}, \mathbf{y} = (\mathbf{y}_1, \mathbf{y}_2)) \in \mathcal{R} \times \mathcal{R}^2 \xleftarrow{\$} \mathcal{S}_{\mathbf{a}}^{3n}(\mathbb{F}_{2^m}) := \text{shake256}(\sigma)$ for the given small rank vector $\mathbf{a}$.
- Compute $\mathbf{s} = (\mathbf{s}_1, \mathbf{s}_2) := \mathbf{x}\mathbf{h} + \mathbf{y} \bmod P$.
- Output $\text{pk} := (\rho, \mathbf{s}), \text{sk} := \mathbf{x}$.

Enc(pk = $(\rho, \mathbf{s}), \mathbf{m} \in \mathbb{F}_{2^m}^k, \theta$):

- Compute $\mathbf{h} \in \mathcal{R}^2 := \text{shake256}(\rho)$ and the generator matrix $\mathbf{G} \in \mathbb{F}_{q^m}^{k \times nn_1}$ of the Augmented Gabidulin code $\mathcal{G}_{\mathbf{g}}^+(nn_1, n', k, m)$.
- Sample $(\mathbf{r}_1, \mathbf{e}, \mathbf{R}_2) \in \mathcal{R}^{n_1} \times \mathcal{R}^{n_1} \times \mathcal{R}^{n_1 \times 2} \xleftarrow{\$} \mathcal{S}_{\mathbf{b}}^{4nn_1}(\mathbb{F}_{q^m}) := \text{shake256}(\theta)$ for the given small rank weight matrix $\mathbf{b}$.
- Compute $\mathbf{u} := \mathbf{h}\mathbf{R}_2^\top + \mathbf{r}_1, \mathbf{v} := \mathbf{s}\mathbf{R}_2^\top + \mathbf{e} + \text{Fold}(\mathbf{m}\mathbf{G})$.
- Output $\mathbf{c} := (\mathbf{u}, \mathbf{v})$.

Dec(sk = $\mathbf{x}, \mathbf{c} = (\mathbf{u}, \mathbf{v})$):

- Output $\mathbf{m} := \mathcal{D}_{\mathcal{G}_{\mathbf{g}}^+}(\text{Unfold}(\mathbf{v} - \mathbf{x}\mathbf{u}))$, where $\mathcal{D}_{\mathcal{G}_{\mathbf{g}}^+}(\cdot)$ denotes the decoding algorithm of the Augmented Gabidulin code $\mathcal{G}_{\mathbf{g}}^+(nn_1, n', k, m)$.

Algorithm 1: BAG-Piglet.PKE

Correctness.

Since $\mathbf{u} = \mathbf{h}\mathbf{R}_2^\top + \mathbf{r}_1$ and $\mathbf{v} = \mathbf{s}\mathbf{R}_2^\top + \mathbf{e} + \text{Fold}(\mathbf{m}\mathbf{G})$, we have

$$\mathbf{v} - \mathbf{x}\mathbf{u} = \text{Fold}(\mathbf{m}\mathbf{G}) + \mathbf{y}\mathbf{R}_2^\top + \mathbf{e} - \mathbf{x}\mathbf{r}_1^\top.$$

To ensure correct decoding, BAG-Piglet employs an augmented Gabidulin code

$$G_{\mathbf{g}}^+(nn_1, n', k, m)$$

over $\mathbb{F}_{2^m}$, which can efficiently correct up to

$$\delta = \left\lfloor \frac{n' - k + \varepsilon}{2} \right\rfloor$$

rank errors. Accordingly, the parameters must satisfy

$$w_R(\mathbf{y}\mathbf{R}_2^\top + \mathbf{e} - \mathbf{x}\mathbf{r}_1^\top) = \sum_{i=1}^2 w_{\mathbf{y}_i} w_{\mathbf{R}_{2,i}} + w_e + w_{\mathbf{x}} w_{\mathbf{r}} \leq \left\lfloor \frac{n' - k + \varepsilon}{2} \right\rfloor,$$

and the decryption failure rate is given by

$$q^{\delta(n' - nn_1)} \sum_{i=0}^{\varepsilon-1} \prod_{j=0}^{i-1} \frac{(q^\delta - q^j)(q^{(nn_1 - n')} - q^j)}{q^i - q^j}.$$

1.4 IND-CCA2 Secure BAG-Piglet.KEM

In this subsection we obtain an IND-CCA2 secure key encapsulation mechanism BAG-Piglet.KEM by applying the salted variant of the Fujisaki–Okamoto transform[10] directly to the public-key encryption scheme BAG-Piglet.PKE.

More specifically, let $\mathcal{E}$ be an instance of the above BAG-Piglet-PKE scheme. Let $\mathcal{G} : \{0, 1\}^* \rightarrow \{0, 1\}^{512}$ and $\mathcal{K} : \{0, 1\}^* \rightarrow \{0, 1\}^{512}$ be two key derivation functions. We instantiate $\mathcal{G}$ with SHA3-512 and $\mathcal{K}$ with SHAKE256. Let $\mathcal{ID}(\cdot)$ be a public-key entropy extraction function. The description of BAG-Piglet.KEM is given in Algorithm 2.

BAG-Piglet. KEM :

KeyGen(1^λ):

- Call $(\text{pk}', \text{sk}') \leftarrow \mathcal{E}.\text{KeyGen}(1^\lambda)$.
- Sample $\xi \xleftarrow{\$} \{0, 1\}^{512}$ uniformly at random.
- Output $\text{pk} := \text{pk}', \text{sk} := (\text{sk}', \text{pk}', \xi)$.

Encap(pk):

- Sample $\mathbf{m} \in \mathbb{F}_{q^m}^k, \text{salt} \in \mathbb{F}_2^{128}$ uniformly at random, and compute $(k, \theta) \leftarrow \mathcal{G}(\mathcal{ID}(\text{pk}), \mathbf{m}, \text{salt})$.
- Call $\text{ct} \leftarrow \mathcal{E}.\text{Enc}(\text{pk}, \mathbf{m}, \theta)$, and derive the session key $K \leftarrow \mathcal{K}(k, \text{ct})$.
- Output ct, K , and salt .

Decap(sk = $(\text{sk}', \text{pk}', \xi)$, ct, salt):

- Call $\mathbf{m}' \leftarrow \mathcal{E}.\text{Dec}(\text{sk}', \text{ct})$.
- Compute $(k', \theta') \leftarrow \mathcal{G}(\mathcal{ID}(\text{pk}'), \mathbf{m}', \text{salt})$.
- Call $\text{ct}' \leftarrow \mathcal{E}.\text{Enc}(\text{pk}', \mathbf{m}', \theta')$.
- If $\text{ct}' = \text{ct}$, compute and output $K' \leftarrow \mathcal{K}(k', \text{ct}')$; otherwise, output $K' \leftarrow \mathcal{K}(\xi, \text{ct}')$ with implicit rejection.

Algorithm 2: BAG-Piglet.KEM

1.5 Parameter Selection

In this subsection, we present the parameter sets of the BAG-Piglet scheme achieving 128, 256, 384, and 512 bits of security, respectively. The specific parameter selections are shown in Table 1.

Random bytes are generated using the `randombytes` function from the OpenSSL library. Specifically, it is used to generate $\rho = \text{seed1}$, $\sigma = \text{seed2}$, and $\theta = \text{seed3}$, and $\mathbf{m}$. Hence, each of the three seeds has size $\lambda/8$ bytes.

We employ the disjoint block errors technique [13] and all blocks have equal length n . In the key generation phase, the error vector $\mathbf{x} \in \mathcal{R}, \mathbf{y} = (\mathbf{y}_1, \mathbf{y}_2) \in \mathcal{R}^2$ and its associated rank vector $w_{(\mathbf{x}, \mathbf{y})}$.

Table 1: BAG-Piglet parameter sets

Instance	q	n	n'	m	n_1	ε	k	$w_{(\mathbf{x}, \mathbf{y})}$	$w_{(\mathbf{r}_1, \mathbf{R}_2, \mathbf{e})}$
BAG-Piglet-128	2	47	43	43	2	30	3	(4, 4, 4)	(2, 3, 3, 3)
BAG-Piglet-256	2	78	78	79	2	54	4	(6, 6, 6)	(3, 3, 4, 4)
BAG-Piglet-384	2	106	103	103	2	77	4	(7, 7, 7)	(4, 4, 4, 4)
BAG-Piglet-512	2	125	130	131	2	92	4	(8, 8, 8)	(4, 4, 5, 5)

In the encryption phase, the error vector $\mathbf{r}_1 \in \mathcal{R}^{n_1}$, $\mathbf{e} \in \mathcal{R}^{n_1}$, $\mathbf{R}_2 \in \mathcal{R}^{2n_1}$ are generated from the `seed3` and its associated rank vector $w_{(\mathbf{r}_1, \mathbf{R}_2, \mathbf{e})}$.

Table 2 provides the polynomials of $P(X)$ and $f(X)$ is the polynomial used to define the finite field $\mathbb{F}_{q^m}$ as $\mathbb{F}_q[X]/\langle f(X) \rangle$.

Table 2: Polynomials considered for BAG-Piglet. $P(X)$ is the polynomial used to define $\mathbb{F}_{q^m}^n$ as $\mathbb{F}_{q^m}[X]/\langle P(X) \rangle$ and $f(X)$ is the polynomial used to define $\mathbb{F}_{q^m}$ as $\mathbb{F}_q[X]/\langle f(X) \rangle$.

Instance	$P(X)$	$f(X)$
BAG-Piglet-128	$X^{43} + X^6 + X^4 + X^3 + 1$	$X^{47} + X^5 + 1$
BAG-Piglet-256	$X^{78} + X^6 + X^5 + X^3 + 1$	$X^{79} + X^9 + 1$
BAG-Piglet-384	$X^{106} + X^{15} + 1$	$X^{103} + X^9 + 1$
BAG-Piglet-512	$X^{125} + X^7 + X^6 + X^5 + 1$	$X^{131} + X^8 + X^3 + X^2 + 1$

Table 3 presents the parameter sizes, including those of the public key, secret key, ciphertext, and shared key. The secret key is generated from a seed. The public key size is given by

$$\left\lceil \frac{2mn}{8} \right\rceil + \lambda/8 \text{ bytes,}$$

and the ciphertext size is

$$\left\lceil \frac{2mnn_1}{8} \right\rceil \text{ bytes.}$$

Table 3: BAG-Piglet parameter sizes

Instance	pk size	ss key	ct size	pk+ct size	shared key
BAG-Piglet-128	522	16	1011	1533	16
BAG-Piglet-256	1573	32	3081	4654	32
BAG-Piglet-384	2778	48	5459	8237	48
BAG-Piglet-512	4158	64	8188	12346	64

Table 4 reports the performance figures, each averaged over 1000 random instances.

Table 4: Millions of CPU cycles of BAG-Piglet.KEM reference implementation.

Instance	KeyGen	Encaps	Decaps
BAG-Piglet-128	0.70	1.62	6.67
BAG-Piglet-256	1.85	4.46	15.59
BAG-Piglet-384	4.65	9.14	26.55
BAG-Piglet-512	9.83	17.87	46.60

2 Design Rationale

2.1 Difficult Problems for Code-based Public-key Cryptography

In this subsection, we present several difficult problems with disjoint-support errors that are used in code-based cryptosystems. These hardness assumptions underpin the security of code-based public-key schemes.

Definition 2.1 (ℓ -RD problem). *Let $\mathbb{N}$ denote the all nonnegative integers. For a positive integer ℓ , let $\mathbf{n} = (n, \dots, n) \in \mathbb{N}^\ell$, and let $\mathbf{t} = (t_1, \dots, t_\ell) \in \mathbb{N}^{\ell \times \ell}$ be a given small associated rank vector. Given a full-rank matrix $\mathbf{G} \in \mathbb{F}_{q^m}^{k \times \ell n}$ and $\mathbf{y} \stackrel{\text{def}}{=} \mathbf{xG} + \mathbf{e}$, where $\mathbf{x} \in \mathbb{F}_{q^m}^k$ is sampled uniformly and $\mathbf{e} \in \mathcal{S}_{\mathbf{t}}^{\ell n}(\mathbb{F}_{q^m})$, the pairwise block rank decoding problem $\ell\text{-RD}(n, k, \mathbf{t})$ asks to recover $\mathbf{x}$ and $\mathbf{e}$.*

Definition 2.2 (ℓ -RSD problem). *Let $\mathbf{n} = (n, \dots, n) \in \mathbb{N}^\ell$ and let $\mathbf{t} = (t_1, \dots, t_\ell) \in \mathbb{N}^\ell$ be a given rank weight vector. Let $\mathbf{H}$ be a parity-check matrix of a random $[\ell n, k]_{q^m}$ -linear code $\mathcal{C}$, and let $\mathbf{s} \in \mathbb{F}_{q^m}^{\ell n - k}$. The pairwise block rank syndrome decoding problem $\ell\text{-RSD}(n, k, \mathbf{t})$ asks to find $\mathbf{e} \in \mathcal{S}_{\mathbf{t}}^{\ell n}(\mathbb{F}_{q^m})$ such that $\mathbf{s} = \mathbf{He}^\top$.*

When $\ell = 1$, The ℓ -RSD problem becomes the *RSD* problem, which has been shown to be NP-hard as it can be probabilistically reduced to the syndrome decoding problem in the Hamming metric [9]. It is well known that the latter is NP-hard [6].

The rank support learning problem [8] can be viewed as a generalization of the RSD problem, where N instances share the same generator matrix (or the same parity-check matrix in the RSD setting), and the errors have a common support.

A $k \times n$ matrix $\mathbf{A} = (a_{ij})$ over $\mathbb{F}_{q^m}$ is said to be *homogeneous* on a support set E if the $\mathbb{F}_q$ -subspace spanned by its entries a_{ij} is equal to E .

Definition 2.3 (ℓ -RSL problem). *Given $(\mathbf{H}, \mathbf{He}^\top)$ and a rank matrix $\mathbf{t}$, where $\mathbf{H} \in \mathbb{F}_{q^m}^{(\ell n - k) \times \ell n}$ is full-rank and $\mathbf{E} = (\mathbf{E}_1, \dots, \mathbf{E}_\ell) \in \mathbb{F}_{q^m}^{N \times \ell n}$. The pairwise block rank support learning problem $\ell\text{-RSL}(n, k, \mathbf{t}, N)$ asks to recover the family of subspaces $(\mathcal{V}_i)_{i \in \{1, \dots, \ell\}}$ so that for each $i \in \{1, \dots, \ell\}$, the error matrix $\mathbf{E}_i \in \mathbb{F}_{q^m}^{N \times n}$ and $\mathcal{V}_i = \text{Supp}(\mathbf{E}_i)$, $\mathbf{E}_i$ is homogeneous on support $\mathcal{V}_i$ with $t_i = w_{\mathbf{E}_i}$ for $1 \leq i \leq \ell$.*

To reduce parameter sizes, many rank-metric cryptosystems based on ideal codes rely on the following structured hardness assumptions.

Definition 2.4. ((ℓ, s) -IRSL problem). For a positive integer ℓ , let $\mathbf{H} \in \mathbb{F}_{q^m}^{(\ell-s)n \times \ell n}$ be the parity-check matrix of an (ℓ, s) -ideal code. Given a syndrome matrix $\mathbf{S} \in \mathbb{F}_{q^m}^{N \times (\ell-t)n}$ and a small rank block vector $\mathbf{t} = (t_1, \dots, t_\ell) \in \mathbb{N}^\ell$, pairwise block ideal rank support learning problem (ℓ, s) -IRSL($n, \mathbf{t}, N$) asks to recover a family of subspaces $\mathcal{V}_1, \dots, \mathcal{V}_\ell \subset \mathbb{F}_{q^m}$ such that there exists a error matrix $\mathbf{E} = (\mathbf{E}_1 \mid \dots \mid \mathbf{E}_\ell)$ with $\text{Supp}(\mathbf{V}_i) = \mathcal{V}_i$, $\mathbf{t} = w_E$, $\mathbf{V}_i$ is homogeneous on $\mathcal{V} - i$ and $\mathbf{H}\mathbf{V}^\top = \mathbf{S}^\top$.

The above problems are referred to as the search version, we also have the decision version of the above problems and omit them here. Furthermore, these variants are also hard if the entries of the given associated rank vector are small.

2.2 Criterion for Parameter Choices

The concrete security of the BAG-Piglet scheme relies on the $(3, 1)$ -DIRSD problem and the $(4, 2)$ -DIRSL problem. There are mainly two classes of attacks against these problems: combinatorial attacks and algebraic attacks (see Section 3 for details). Therefore, parameter selection must ensure resistance against both types of attacks.

In the following we consider the parameters in BAG-Piglet scheme needed to satisfy the detailed conditions in detail:

- m should be prime in order to avoid algebraic attack.
- BAG-Piglet uses augmented Gabidulin code $G_{\mathbf{g}}^+(nn_1, n', k, m)$ as an auxiliary code, which is deterministic decoding algorithm, and is able to correct $\leq \left\lfloor \frac{n' - k + \varepsilon}{2} \right\rfloor$ rank errors at least with decoding failure rate

$$\text{DFR} = q^{\delta(n' - n)} \sum_{i=0}^{\varepsilon-1} \prod_{j=0}^{i-1} \frac{(q^\delta - q^j)(q^{(n-n')} - q^j)}{q^j - q^j}$$

where $\varepsilon \in \{1, 2, \dots, \min(nn_1 - n', n' - k)\}$.

- For the augmented Gabidulin code $G_{\mathbf{g}}^+(nn_1, n', k, m)$, i.e., $n' \leq m$ is a positive integer, and n, n_1 are positive integers, satisfying $0 < k < n'$, where k is the dimension of the augmented Gabidulin code and so $2^{km} \geq 2^\lambda$. It must satisfy the decoding capability requirement of the augmented Gabidulin code:

$$w - \mathbf{x}\mathbf{r}_1 + \mathbf{y}_1\mathbf{R}_{2,1} + \mathbf{y}_2\mathbf{R}_{2,2} + \mathbf{e} \leq \left\lfloor \frac{n' - k + \varepsilon}{2} \right\rfloor.$$

- The security of the public-key generation phase $\mathbf{s} = \mathbf{x}\mathbf{H} + \mathbf{y}$ can be reduced to the $(3, 1)$ -IRSD($n, w_{(\mathbf{x}, \mathbf{y})}$) difficult problem, i.e.,

$$(\mathbf{I}_n \mid \mathbf{h}_1 \mid \mathbf{h}_2) \begin{pmatrix} \mathbf{y}^\top \\ \mathbf{x}_1^\top \\ \mathbf{x}_2^\top \end{pmatrix} = \begin{pmatrix} \mathbf{s}_1^\top \\ \mathbf{s}_2^\top \end{pmatrix}, \quad (1)$$

which must satisfy the best know attacks discussed in Section 3 such as OJ attacks, PRR attacks, MM algebraic attacks.

- The security of the encryption phase can be reduced to the $(4, 2)$ -IRSL($n, w_{(\mathbf{r}_1, \mathbf{e}, \mathbf{R}_2)}, n_2$) difficult problem, i.e.,

$$\begin{pmatrix} \mathbf{u} \\ \mathbf{v} - \text{Fold}(\mathbf{m}\mathbf{G}) \end{pmatrix} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \mathbf{h}_1 & \mathbf{h}_2 \\ \mathbf{0} & \mathbf{I}_n & \mathbf{s}_1 & \mathbf{s}_2 \end{pmatrix} \begin{pmatrix} \mathbf{r}_1^\top \\ \mathbf{e}^\top \\ \mathbf{R}_{2,1}^\top \\ \mathbf{R}_{2,2}^\top \end{pmatrix}, \quad (2)$$

which must satisfy the best know attacks discussed in Section 3 such as OJ attacks, PRR attacks, MM algebraic attacks.

2.3 Augmented Gabidulin Codes and Decoding

The Augmented Gabidulin code is to append a string of zeros at the end of a Gabidulin code; see [7] for details. Its definition and some properties are put in notations and so are omit here.

In [7] gives a decoding method for the Augmented Gabidulin code $\mathcal{G}_{\mathbf{g}}^+(n, n', k, m)$, in which $(n, n', k, m) \in \mathbb{N}^4$ satisfy $k \leq n' \leq m \leq n$ and $\mathbf{g} = (g_1, \dots, g_{n'}) \in \mathbb{F}_{q^m}^{n'}$ satisfy $w_R(\mathbf{g}) = n'$, and let $\bar{\mathbf{g}} \stackrel{\text{def}}{=} (\mathbf{g} \mid \mathbf{0}_{n-n'}) \in \mathbb{F}_{q^m}^n$. However, it requires solving a linear system with n' equations and $k + 2\delta - \varepsilon$ unknowns over $\mathbb{F}_{q^m}$, which consumes a large amount of storage and is inefficient. Therefore, we give a new fast decoding algorithm below.

Assume that a noisy codeword $\bar{\mathbf{y}} = \bar{\mathbf{c}} + \bar{\mathbf{e}} \in \mathbb{F}_{q^m}^n$ is given, where $\bar{\mathbf{c}} = \mathbf{x}\mathbf{G}$ belongs to $\mathcal{G}_{\mathbf{g}}^+(n, n', k, m)$, and $w_{\mathbf{e}} \leq \delta$. By construction, $\bar{\mathbf{c}} = (* \mid 0 \dots 0)$, and therefore the last $n - n'$ coordinates of $\bar{\mathbf{y}}$ are exactly the last $n - n'$ entries of $\bar{\mathbf{e}}$.

Let $\mathbf{y}, \mathbf{c}, \mathbf{e}$ denote the vectors generated by the first n' coordinates of $\bar{\mathbf{y}}, \bar{\mathbf{c}}, \bar{\mathbf{e}}$, respectively. Then

$$y_i = f(g_i) + e_i, \quad 1 \leq i \leq n'.$$

Assume that the dimension of the support of the last $n - n'$ entries of the error vector $\bar{\mathbf{e}}$ is ε . Let E_2 denote the vector space generated by these elements, and let V_2 be the unique monic q -polynomial annihilating the elements of E_2 . Then

$$V_2(y_i) = V_2 \circ f(g_i) + V_2(e_i), \quad 1 \leq i \leq n'. \quad (*)$$

Construct a new Gabidulin code $\mathcal{G}(\mathbf{g}, n', \varepsilon + k)$. It is straightforward that this code has minimum distance

$$d' = n' - \varepsilon + k + 1,$$

and therefore it can correct at most $\delta - \varepsilon$ errors.

Lemma 2.1. *The Gabidulin code $\mathcal{G}(\mathbf{g}, n', \varepsilon + k)$ has minimum distance*

$$d' = n' - \varepsilon + k + 1,$$

and can correct at most $\delta - \varepsilon$ errors.

Proof. By the properties of Gabidulin codes, one directly obtains that the minimum distance is $d' = n' - \varepsilon + k + 1$. Moreover,

$$\delta = \left\lfloor \frac{n' - k + \varepsilon}{2} \right\rfloor = \begin{cases} \frac{n' - k + \varepsilon}{2}, & \text{if } n' - k + \varepsilon \text{ is even,} \\ \frac{n' - k + \varepsilon - 1}{2}, & \text{if } n' - k + \varepsilon \text{ is odd.} \end{cases}$$

Hence, we have $\left\lfloor \frac{n' - k - \varepsilon}{2} \right\rfloor = \delta - \varepsilon$. $\square$

In $(*)$, the vector

$$(V_2 \circ f(g_1), V_2 \circ f(g_2), \dots, V_2 \circ f(g_{n'}))$$

can be viewed as a codeword of the Gabidulin code $\mathcal{G}(\mathbf{g}, n', \varepsilon + k)$, while $(V_2(e_1), \dots, V_2(e_{n'}))$ has rank at most $\delta - \varepsilon$. Therefore, by applying the decoding algorithm for the Gabidulin code $\mathcal{G}(\mathbf{g}, n', \varepsilon + k)$ to the received word $(V_2(y_1), \dots, V_2(y_{n'}))$, one obtains the corresponding codeword $(V_2 \circ f(g_1), V_2 \circ f(g_2), \dots, V_2 \circ f(g_{n'}))$, and then f can be recovered by the left Euclidean algorithm.

Theorem 2.1. *The complexity of the decoding algorithm for $\mathcal{G}_{\mathbf{g}}^+(n, n', k, m)$ is $O(n^2)$ operations over $\mathbb{F}_{q^m}$.*

Proof. We give the cost of each steps in decoding procedure: computing the rank weight of $\mathbf{y}$ needs $(n - n') \cdot \varepsilon$ operations in $\mathbb{F}_{q^m}$, finding the vanishing polynomial needs ε^2 operations, acting the first part of $\mathbf{y}$ needs $n' \cdot \varepsilon$, running the decoding algorithm of Gabidulin code needs n'_2 and left dividing vanishing polynomial needs $k \cdot \varepsilon$ operations in $\mathbb{F}_{q^m}$. So the total complexity of decoding the Augmented Gabidulin code is required. $\square$

2.4 Statement of Design

Compared to the original Piglet-1 [14], BAG-Piglet introduces the following improvements:

- **Structural simplification.** The public key $\mathbf{h}$ in $\mathbf{s} = \mathbf{xh} + \mathbf{y}$ is reduced from a 2-dimensional matrix to a 2-dimensional vector, resulting in a cleaner design.
- **Computational efficiency.** The number of finite field multiplications per encryption is reduced from 18 to 12.
- **Noise sampling strategy.** In Piglet-1, errors for the key generation (or encryption) are sampled from the same supports. BAG-Piglet adopts the disjoint block error technique from [13], which significantly reduces the parameter size.
- **Parameter size.** As a result of the above improvements, total size (pk + ct) is reduced from $8mn/8$ to $6mn/8$.

3 Security Analysis

3.1 Theoretical Security Proof

In this subsection, we first prove the IND-CPA security of our encryption scheme (BAG-Piglet.PKE) under the DIRSD and DIRSL assumptions. The security of the KEM/DEM version (BAG-Piglet.KEM) is then provided by the salted variant of the Fujisaki–Okamoto transformation, and the tightness of the reduction is discussed as detailed in [10].

IND-CPA Security for BAG-Piglet.PKE under ROM.

Theorem 3.1. *The scheme presented in Algorithm 1 is IND-CPA under the $(3, 1)$ -DIRSD $(n, w_{(\mathbf{x}, \mathbf{y})})$ and $(4, 2)$ -DIRSL $(n, w_{(\mathbf{r}_1, \mathbf{e}, \mathbf{R}_2)}, n_2)$ assumptions.*

Proof. The proof is similar to that in [2], with the difference being that the difficult problems are the $(3, 1)$ -DIRSD $(n, w_{(\mathbf{x}, \mathbf{y})})$ and $(4, 2)$ -DIRSL $(n, w_{(\mathbf{r}_1, \mathbf{e}, \mathbf{R}_2)}, n_2)$ instances, defined as follows:

$$(\mathbf{I}_n \ \mathbf{h}_1 \ \mathbf{h}_2) \begin{pmatrix} \mathbf{y}^\top \\ \mathbf{x}_1^\top \\ \mathbf{x}_1^\top \end{pmatrix} = \begin{pmatrix} \mathbf{s}_1^\top \\ \mathbf{s}_2^\top \end{pmatrix}, \begin{pmatrix} \mathbf{u} \\ \mathbf{v} - \text{Fold}(\mathbf{m}\mathbf{G}) \end{pmatrix} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \mathbf{h}_1 & \mathbf{h}_2 \\ \mathbf{0} & \mathbf{I}_n & \mathbf{s}_1 & \mathbf{s}_2 \end{pmatrix} \begin{pmatrix} \mathbf{r}_1^\top \\ \mathbf{e}^\top \\ \mathbf{R}_{2,1}^\top \\ \mathbf{R}_{2,2}^\top \end{pmatrix}.$$

The following is a sketch of the proof. We employ a hybrid argument: starting from the real encryption of $\mathbf{m}_0$, a sequence of games gradually transitions to that of $\mathbf{m}_1$. Each adjacent indistinguishability is reduced to the $(3, 1)$ -DIRSD or $(4, 2)$ -DIRSL problem.

Game $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^1(\lambda)$ is the real game with challenge message $\mathbf{m}_0$.

Game $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^2(\lambda)$ replaces the correctly generated $\mathbf{s} = \mathbf{x}\mathbf{h} + \mathbf{y}$ in $\mathbf{pk}$ with a uniformly random vector. This corresponds to a $(3, 1)$ -DIRSD $(n, w_{(\mathbf{x}, \mathbf{y})})$ instance, as

$$(\mathbf{I}_n \ \mathbf{h}_1 \ \mathbf{h}_2) \begin{pmatrix} \mathbf{y}^\top \\ \mathbf{x}_1^\top \\ \mathbf{x}_1^\top \end{pmatrix} = \begin{pmatrix} \mathbf{s}_1^\top \\ \mathbf{s}_2^\top \end{pmatrix},$$

with $(\mathbf{I}_n \ \mathbf{h}_1 \ \mathbf{h}_2)$ is the parity-check matrix of a random linear code over $\mathbb{F}_{q^m}$ and $\mathbf{x}, \mathbf{y}$ are sampled with low associated rank. Distinguishing $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^1(\lambda)$ from $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^2(\lambda)$ with advantage ϵ reduces to solving the $(3, 1)$ -DIRSD $(n, w_{(\mathbf{x}, \mathbf{y})})$ problem with the same advantage. We have thus

$$|\Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^2(\lambda) = 1] - \Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^1(\lambda) = 1]| = \text{Adv}_{(n, w_{(\mathbf{x}, \mathbf{y})})}^{(3,1)\text{-DIRSD}}(\lambda).$$

Game $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^3(\lambda)$ replaces the ciphertext components $\mathbf{u} = \mathbf{h}\mathbf{R}_2^\top + \mathbf{r}_1$ and $\mathbf{v} = \mathbf{s}\mathbf{R}_2^\top + \mathbf{e} + \text{Fold}(\mathbf{m}_0\mathbf{G})$ with uniformly random values. This corresponds to a $(4, 2)$ -DIRSL $(n, w_{(\mathbf{r}_1, \mathbf{e}, \mathbf{R}_2)}, n_2)$ instance, as

$$\begin{pmatrix} \mathbf{u} \\ \mathbf{v} - \text{Fold}(\mathbf{m}\mathbf{G}) \end{pmatrix} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \mathbf{h}_1 & \mathbf{h}_2 \\ \mathbf{0} & \mathbf{I}_n & \mathbf{s}_1 & \mathbf{s}_2 \end{pmatrix} \begin{pmatrix} \mathbf{r}_1^\top \\ \mathbf{e}^\top \\ \mathbf{R}_{2,1}^\top \\ \mathbf{R}_{2,2}^\top \end{pmatrix},$$

with $(\mathbf{h}_1 \parallel \mathbf{h}_2)$ is the parity-check matrix of an ideal code, $(\mathbf{s}_1 \parallel \mathbf{s}_2)$ is the syndrome matrix and $\mathbf{r}_1, \mathbf{e}, \mathbf{R}_2$ are randomly sampled low-rank-weight matrices with the rank weight matrix $w_{(\mathbf{r}_1, \mathbf{e}, \mathbf{R}_2)}$. Again, distinguishing $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^2(\lambda)$ from $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^3(\lambda)$ with advantage ϵ reduces to solving the $(4, 2)$ -DIRSL $(n, w_{(\mathbf{r}_1, \mathbf{e}, \mathbf{R}_2)}, n_2)$ problem with the same advantage. Thus

$$|\Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^3(\lambda) = 1] - \Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^2(\lambda) = 1]| = \text{Adv}_{(n, w_{(\mathbf{r}_1, \mathbf{e}, \mathbf{R}_2)}, n_2)}^{(4, 2)\text{-DIRSL}}(\lambda).$$

Game $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^4(\lambda)$ encrypts $\mathbf{m}_1$ instead of $\mathbf{m}_0$ using freshly sampled randomness. The output distributions of $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^3(\lambda)$ and $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^4(\lambda)$ are identical: for any $(\mathbf{r}_1, \mathbf{e}, \mathbf{R}_2)$ in $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^3(\lambda)$ producing $(\mathbf{u}, \mathbf{v})$, the mapping $(\mathbf{r}'_1, \mathbf{e}', \mathbf{R}'_2)$ (with $\mathbf{r}'_1 = \mathbf{r}_1$, $\mathbf{R}'_2 = \mathbf{R}_2$, and $\mathbf{e}' = \mathbf{e} + \text{Fold}(\mathbf{m}_0 \mathbf{G}) - \text{Fold}(\mathbf{m}_1 \mathbf{G})$) is a bijection to a valid tuple in $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^4(\lambda)$ producing the same ciphertext. Then, we have

$$|\Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^4(\lambda) = 1] - \Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^3(\lambda) = 1]| = 0.$$

Games $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^5(\lambda)$ and $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^6(\lambda)$ mirror $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^2(\lambda)$ and $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^1(\lambda)$, respectively, except that the challenge message $\mathbf{m}_1$ is encrypted instead of $\mathbf{m}_0$. By the similar reductions, we have

$$\begin{aligned} |\Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^5(\lambda) = 1] - \Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^4(\lambda) = 1]| &= \text{Adv}_{(n, w_{(\mathbf{r}_1, \mathbf{e}, \mathbf{R}_2)}, n_2)}^{(4, 2)\text{-DIRSL}}(\lambda), \\ |\Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^6(\lambda) = 1] - \Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^5(\lambda) = 1]| &= \text{Adv}_{(n, w_{(\mathbf{x}, \mathbf{y})})}^{(3, 1)\text{-DIRSD}}(\lambda). \end{aligned}$$

Putting all these together, the advantage of an adversary $\mathcal{A}$ against the IND-CPA experiment is bounded as

$$\begin{aligned} \text{Adv}_{\text{BAG-Piglet.PKE}, \mathcal{A}}^{\text{CPA}}(\lambda) &= |\Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^6(\lambda) = 1] - \Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^1(\lambda) = 1]| \\ &\leq 2 \cdot (\text{Adv}_{(n, w_{(\mathbf{x}, \mathbf{y})})}^{(3, 1)\text{-DIRSD}}(\lambda) + \text{Adv}_{(n, w_{(\mathbf{r}_1, \mathbf{e}, \mathbf{R}_2)}, n_2)}^{(4, 2)\text{-DIRSL}}(\lambda)) \\ &\leq \text{negl}(\lambda) \end{aligned}$$

for negl a negligible function. This completes the proof. $\square$

IND-CCA Security for BAG-Piglet.KEM under ROM. The BAG-Piglet.PKE scheme satisfies IND-CPA security and has a sufficiently low decryption failure probability (bounded by $2^{-\lambda}$). By applying the salted variant of the Fujisaki–Okamoto transformation, the IND-CPA security of our BAG-Piglet.PKE tightly implies IND-CCA2 security of the resulting KEM version shown in Algorithm 2 (BAG-Piglet.KEM). We adopt the SFO [PKE, G, H, len_{salt}] transformation algorithm proposed in [10]. In the Random Oracle (RO) model, this multi-user transformation admits the following security reduction:

Lemma 3.1 (SFO Transformation in RO Model [10]). *Let PKE be a public-key encryption scheme, and let KEM be the KEM constructed as $\text{KEM} := \text{SFO}[\text{PKE}, \text{G}, \text{H}, \text{len}_{\text{salt}}]$. If PKE is $\delta(n_u)$ -correct, then KEM is $\delta(n_u)$ correct in the random oracle model. Let the number of queries to ChALL_j be bounded by n_c and $\mathcal{L}_{S_j}$ represent the set of salt values sampled by during those queries. Consider a $\text{IND}_{n_c, n_u}\text{-CPA}$ adversary $\mathcal{B}$ against KEM, issuing at most q_H queries to H , q_G queries to G , and let $q = (q_H + q_G)$. Suppose that no*

salt appears in any $\mathfrak{L}_{M_j}$ more than t times. There exists an IND_{n_c, n_u} -CPA adversary $\mathcal{A}$ against PKE such that

$$\text{Adv}_{\text{KEM}}^{\text{IND}_{n_c, n_u}\text{-CPA}}(\mathcal{B}) \leq \frac{n_u n_c (n_c - 1)}{|\mathcal{M}| \cdot 2^{\text{len}_{\text{salt}}}} + \frac{2t(q_H + q_G)}{|\mathcal{M}|} + 2 \cdot \text{Adv}_{\text{PKE}}^{\text{IND}_{n_c, n_u}\text{-CPA}}(\mathcal{A}),$$

where n_u denotes the number of users, n_c denotes the number of challenge ciphertexts, G, H are hash functions, and $|\mathcal{M}|$ represents the plaintext space size of PKE. Furthermore $\mathcal{A}$ and $\mathcal{B}$ have similar run-time.

Apparently, by the above lemma, BAG-Piglet.KEM transformed from the IND-CPA secure BAG-Piglet.PKE scheme is an IND-CCA2 secure key encapsulation mechanism.

IND-CCA Security for BAG-Piglet.KEM under QROM. Since classical computers are special cases of quantum computers, the aforementioned reduction naturally holds in the quantum model. Therefore, we provide no additional reduction proof in this section.

Our construction employs the SFO [PKE, $G, H, \text{len}_{\text{salt}}$] transformation proposed in [10]. In the multi-user setting and under the quantum random oracle model (QROM), this transformation yields the following reduction:

Lemma 3.2 (SFO Transformation in QRO Model [10]). *Let PKE be a public-key encryption scheme, and let KEM be the KEM constructed as $\text{KEM} := \text{SFO}[\text{PKE}, G, H, \text{len}_{\text{salt}}]$. If PKE is $\delta(n_u)$ -correct, then KEM is $\delta(n_u)$ correct in the random oracle model. Let the number of queries to CHALL_j at index $j \in [n_u]$ be bounded by n_c and let $\mathfrak{L}_{S_j}$ represent the set of salt values sampled during those queries. Consider a IND_{n_c, n_u} -CPA adversary $\mathcal{B}$ against KEM, issuing at most q_H and q_G many queries in total to H and G respectively, with a total depth of d . Suppose that no salt appears in any $\mathfrak{L}_{M_j}$ more than t times.*

Furthermore, assume that $\mathcal{B}$ does not query $e\text{CO.Ext}$ on any of its challenge ciphertexts. Then there exists a quantum IND_{n_c, n_u} -CPA adversary $\mathcal{A}$ against PKE such that

$$\begin{aligned} \text{Adv}_{\text{KEM}}^{\text{IND}_{n_c, n_u}\text{-CPA}}(\mathcal{B}) &\leq 4\sqrt{d \cdot \text{Adv}_{\text{PKE}}^{\text{IND}_{n_c, n_u}\text{-CPA}} + \frac{n_u n_c (n_c - 1)}{|\mathcal{M}| \cdot 2^{\text{len}_{\text{salt}}}}} \\ &\quad + 4\sqrt{\frac{dt(q_H + q_G)}{|\mathcal{M}|}} \end{aligned}$$

where n_u denotes the number of users, n_c denotes the number of challenge ciphertexts, G, H are hash functions, and $|\mathcal{M}|$ represents the plaintext space size of PKE.

3.2 Security Analysis on Known Attacks

The most effective attacks on BAG-Piglet scheme are combinatorial attacks and algebraic attacks. We adapt the Ourivski-Johansson (OJ for short) and PRR combinatorial attacks [12] and MaxMinors (MM for short) attacks on IRSD problem with $\ell 1$ -errors (pairwise disjoint errors) into $(\ell, 2)$.

In addition, since the number of syndrome in $(4, 2)$ -IRSL $(n, \mathbf{t}, n_1)$ is $n_1 = 2$, which is small, we only consider the security of $(4, 2)$ -IRSD $(n, \mathbf{t})$ problem.

The above attacks are usually made on the following model.

Model 1. Let $\beta_1, \dots, \beta_m$ be a basis of $\mathbb{F}_{q^m}$ over $\mathbb{F}_q$. In the above attacks, the RSD problem is the dual of the rank decoding problem, which is reduced to finding a codeword of rank weight t in the code $\mathcal{C}_y = \mathcal{C} \oplus \langle \mathbf{y} \rangle_{\mathbb{F}_{q^m}}$. The error vector satisfies the equation

$$\mathbf{e} \mathbf{H}_y^T = \mathbf{0},$$

where $\mathbf{H}_y \in \mathbb{F}_{q^m}^{(N-K-1) \times N}$ is a systematic parity-check matrix for $\mathcal{C}_y$. Then we write the coefficient matrix of $\mathbf{e}$ under the basis β into the product $\mathbf{S}\mathbf{C}$, where $\mathbf{S} \in \mathbb{F}_{q^m}^{m \times t}$ and $\mathbf{C} \in \mathbb{F}_q^{t \times N}$ are the support and coefficient matrices respectively.

3.2.1 O-J Combinatorial Attacks

We give the complexity of the OJ attack for $(4, 2)$ -IRSD problem. In this scheme, take $\omega = 2.8$ as the Strassen constant.

Theorem 3.2. For the $(4, 2)$ -IRSD $(n, \mathbf{b})$ and $\mathbf{b} = (t_1, t_2, t_3, t_4)$, with $t_1 \leq t_2 \leq t_3 \leq t_4$ and $t = t_1 + t_2 + t_3 + t_4$, the complexity of OJ attack is

$$\mathcal{O}((m(t-1) + (n-t_1) + (n-t_2))^\omega q^{(n-t_1)(t_1-1) + (n-t_2)(t_2-1) + t_4}).$$

Proof. We write the coefficient matrix of $\mathbf{e}$ under the basis β into the product $\mathbf{S}\mathbf{C}$ in Model 1, where $\mathbf{S}$ and $\mathbf{C}$ are the support and coefficient matrices in the following form respectively. under the given $(4, 2)$ -block errors,

$$\mathbf{S} = \left(\begin{array}{c|c} 1 & S' \end{array} \right), \quad \mathbf{C} = \left(\begin{array}{c|c|c|c} \mathbf{I}_{t_1} \mathbf{A}_1 & 0 & 0 & 0 \\ \hline 0 & \mathbf{I}_{t_2} \mathbf{A}_2 & 0 & 0 \\ \hline 0 & 0 & \mathbf{I}_{t_3} \mathbf{A}_3 & 0 \\ \hline 0 & 0 & 0 & \mathbf{I}_{t_4} \mathbf{A}_4 \end{array} \right),$$

where and $S' \in \mathbb{F}_q^{m \times (t-1)}$, $C'_1 \in \mathbb{F}_q^{g \times (n-t_1)}$.

Let $\mathbf{S} = \left(\begin{array}{c|c} 1 & \mathbf{s}' \end{array} \right) \in \mathbb{F}_q^{m \times t}$ where $\mathbf{s}' = S_{\{1\}, \{2..t\}} \in \mathbb{F}_q^{1 \times (t-1)}$ and $\mathbf{S}'' = \mathbf{S}_{\{2..m\}, \{2..t\}} \in \mathbb{F}_q^{(m-1) \times (t-1)}$.

Let $k = 2n$. Splitting C into $\mathcal{A}_1 = \left(\begin{array}{c|c} 1 & \mathbf{c}^* \end{array} \right) \in \mathbb{F}_q^{t \times (2n+1)}$ and $\mathcal{A}_2 = \left(\begin{array}{c} \mathbf{0}_{1 \times (2n-1)} \\ \mathbf{C}_2^* \end{array} \right) \in \mathbb{F}_q^{t \times (2n-1)}$, where $\mathbf{c}^* = C_{\{1\}, \{2..2n+1\}} \in \mathbb{F}_q^{2n}$, $\mathbf{C}_1^* = C_{\{2..t\}, \{2..2n+1\}} \in \mathbb{F}_q^{(t-1) \times 2n}$. For $\mathcal{A}_2$, let $\mathbf{a}_j$ be the j -th column of $\mathbf{A}_2$ and $\mathbf{c}_2^*(j)$ be the j -th column of $\mathbf{C}_2^*$ for $j \in \{1..2n-1\}$.

Then

$$(\mathcal{A}_1 \mathbf{a}_j) = \left(\begin{array}{c|c|c} 1 & \mathbf{c}^* & \mathbf{0} \\ \hline \mathbf{0}_{(t-1) \times 1} & \mathbf{C}_1^* & \mathbf{c}_2^*(j) \end{array} \right)$$

and

$$\begin{aligned} S(\mathcal{A}_1 \mathbf{a}_j) \cdot T_j &= \left(\begin{array}{c|c} 1 & \mathbf{s}' \\ \hline \mathbf{0}_{(m-1) \times 1} & \mathbf{S}'' \end{array} \right) \cdot \left(\begin{array}{c|c|c} 1 & \mathbf{c}^* & \mathbf{0} \\ \hline \mathbf{0}_{(t-1) \times 1} & \mathbf{C}_1^* & \mathbf{c}_2^*(j) \end{array} \right) \cdot T_j \\ &= \left(\begin{array}{c|c|c} 1 & \mathbf{c}^* + \mathbf{s}' \mathbf{C}_1^* & \mathbf{s}' \mathbf{c}_2^*(j) \\ \hline \mathbf{0}_{(t-1) \times 1} & \mathbf{S}'' \mathbf{C}_1^* & \mathbf{S}'' \mathbf{c}_2^*(j) \end{array} \right) \cdot T_j = \mathbf{0}_{m \times m}. \end{aligned}$$

The above equation gives a quadratic multivariate system with m^2 quadratic polynomials and at most $m(t-1) + (n-t_1)t_1 + (n-t_2)t_2 + t_4$ variables.

By first guess the entries of $\mathbf{C}_1^*$ and $\mathbf{c}_2^*(j)$, we get a linear equation system $\mathbf{S}(\mathcal{A}_1 \mathbf{a}_j) \mathbf{T}_j = \mathbf{0}$ about the entries of $\mathbf{S}$ with m^2 equations and $m(t-1) + n - t_1 + n - t_2$ unknowns. Since there are at most $q^{(n-t_1)(t_1-1) + (n-t_2)(t_2-1) + t_4}$ numbers of $\mathbf{C}_1^*$ and $\mathbf{c}_2^*(j)$ and so the total complexity is required. $\square$

For the (3, 1)-IRSD problem, we use the result in [13] as follows.

For the (3, 1)-IRSD $(n, \mathbf{t} = (t_1, t_2, t_3))$ with $t_1 \leq t_2 \leq t_3$ and $t = t_1 + t_2 + t_3 - 2g$, the complexity of OJ attack is

$$\mathcal{O}((m(t-1) + (n-t_1))^\omega q^{(n-t_1)(t_1-1)+t_3}).$$

3.2.2 PRR Combinatorial Attacks

In [12], authors introduced a sum-rank metric and presented a kind of attack on the SD problem under the sum-rank weight, called PRR attacks, which can be used to attack RSD with disjoint block errors. For BAG-Piglet, we provide the complexity of PRR attack on the (ℓ, s) -IRSD $(n, \mathbf{t})$ problem. over $\mathbb{F}_{q^m}$ where the given associated rank vector $\mathbf{t} = (t_1, \dots, t_\ell)$ and $t = t_1 + \dots + t_\ell$, which can be regarded the instance of $N = \ell n, K = (\ell - s)n + 1$ in Model 1.

We follow the framework of the PRR generic decoder [12], which is a Las Vegas-type randomized algorithm consisting of two main steps:

1. Randomly choose a super-support $F_i \supseteq \text{Supp}(\mathbf{e}_i)$ of dimension $r_i \geq t_i$ for each block.
2. If the chosen super-supports contain the true error supports, recover the error via erasure decoding.

The expected complexity is the product of the cost of one iteration and the expected number of iterations.

Case 1: Column Support Strategy ($n \geq m$). When $n \geq m$, the attack uses column supports, where all supports lie in the common ambient space $\mathbb{F}_q^m$. The attacker chooses three private subspaces $P_i \subseteq \mathbb{F}_q^m$ of dimension r_i , yielding a total super-support $F = P_1 + \dots + P_\ell$ of dimension $r = r_1 + \dots + r_\ell$.

The probability that a randomly chosen r -dimensional subspace F contains the true error support $E = \text{Supp}(\mathbf{e}_1) + \dots + \text{Supp}(\mathbf{e}_\ell)$ of dimension t is

$$\Pr(\text{success}) = \frac{\binom{m-t}{r-t}}{\binom{m}{r}} \cdot (\text{block-structure correction}).$$

Using the standard Gaussian binomial estimates $\binom{m}{r} \approx q^{r(m-r)}$, this probability is asymptotically

$$\Pr(\text{success}) \approx q^{-\sum_{i=1}^{\ell} t_i(m-r_i)+m}.$$

Hence, the expected number of iterations is

$$\mathbb{E}[\#\text{iterations}] = \Pr(\text{success})^{-1} \approx 2^{\sum_{i=1}^{\ell} t_i(m-r_i)-m}.$$

For erasure decoding to yield a unique solution, the system must have at least as many equations as unknowns. The number of equations is $m(N - K) = m(\ell - s - 1)n$, and the number of unknowns is $n \sum_{i=1}^{\ell} r_i$. Thus we require

$$n \sum_{i=1}^{\ell} r_i \leq m(\ell - s - 1)n.$$

Additionally, the total super-support cannot exceed the ambient space:

$$r_1 + \dots + r_\ell \leq m.$$

The cost of one iteration is dominated by Gaussian elimination on a system of size $mn(\ell - s - 1)$, which costs $\mathcal{O}((m(\ell - s - 1)n)^\omega)$ operations, where $2 \leq \omega \leq 3$ is the linear algebra constant. Therefore, the total complexity of the PRR attack in the column support case is

$$\mathcal{O}\left((m(\ell - s - 1)n)^\omega \cdot 2^{\sum_{i=1}^{\ell} t_i(m-r_i)-m}\right)$$

subject to the constraints

$$\begin{cases} t_i \leq r_i \leq m, & i = 1, \dots, \ell, \\ r_1 + \dots + r_\ell \leq m, \\ n \sum_{i=1}^{\ell} r_i \leq m(\ell - s - 1)n, \end{cases}$$

and where $(r_1, \dots, r_\ell)$ are chosen to minimize the exponent $\sum_{i=1}^{\ell} t_i(m - r_i) - m$.

Case 2: Row Support Strategy ($n \leq m$) .

When $n \leq m$, the attack uses row supports, where each block's support lies in its own space $\mathbb{F}_q^n$. In this setting, the private subspaces are chosen independently in each

block. The total support size is constrained by the dimension of the nullspace of the parity-check matrix:

$$r_1 + \dots + r_\ell \leq (\ell - s - 1)n.$$

The success probability is now

$$\Pr(\text{success}) \approx q^{-\sum_{i=1}^{\ell} t_i(n-r_i)},$$

where the factor $-m$ from the column-support case disappears because the supports are chosen in different spaces and there is no global $\binom{m}{r}$ denominator.

The cost of one iteration is $\mathcal{O}((m^2((\ell - s - 1)n)^\omega)$, as the system must be expanded from $\mathbb{F}_{q^m}$ to $\mathbb{F}_q$.

Thus, the total complexity in the row support case is

$$\mathcal{O}\left(m^2((\ell - s - 1)n)^\omega \cdot 2^{\sum_{i=1}^{\ell} t_i(n-r_i)}\right)$$

subject to

$$\begin{cases} t_i \leq r_i \leq n, & i = 1, \dots, \ell, \\ r_1 + \dots + r_\ell \leq (\ell - s - 1)n, \end{cases}$$

with $(r_1, \dots, r_\ell)$ chosen to minimize the exponent $\sum_{i=1}^{\ell} t_i(n - r_i)$.

3.2.3 Algebraic Attacks

In this section, we provide the MaxMinors (MM) algebraic attack on the IRSD problem with disjoint errors.

For the $(3, 1)$ -IRSD($n, \mathbf{t}$) problem, we use the result in [3] as follows.

Proposition 3.1. *For the $(3, 1)$ -IRSD($n, \mathbf{t}$) problem over $\mathbb{F}_{q^m}$ where $\mathbf{t} = (t_1, \dots, t_3)$, $t_1 \leq t_2 \leq t_3$, $t = t_1 + t_2 + t_3$, define*

$$N_3(n, \mathbf{t}) = \sum_{j=t_2-1}^{t_1+t_2} \binom{n-1}{j} \binom{n}{t-j}$$

and

$$T_3(\mathbf{n}, \mathbf{t}) = \binom{n_1}{t_1} \binom{n_2}{t_2} \binom{n_3}{t_3},$$

where $\mathbf{n} = (n_1, n_2, n_3)$.

If $mN_3(n, \mathbf{t}) \geq T_3(\mathbf{n}, \mathbf{t}) - 1$ with $\mathbf{n} = (n, n, n)$, then the cost of MM attack to the hard problem is

$$\mathcal{O}(mN_3(n, \mathbf{a})(T_3(\mathbf{n}, \mathbf{t}))^{\omega-1}),$$

otherwise, the cost of hybrid approach is

$$\mathcal{O}\left(\min_{\substack{\mathbf{a}=(a_1, a_2, a_3) \\ mN_3(n, \mathbf{t}) \geq T_3(\mathbf{n}-\mathbf{a}, \mathbf{t})-1}} q^{a_1 t_1 + a_2 t_2 + a_3 t_3} \cdot mN_3(n, \mathbf{t})(T_3(\mathbf{n}-\mathbf{a}, \mathbf{t}))^{\omega-1}\right).$$

We now extend the result to the $[4, 2]$ -IRSD problem by adopting the similar method in [3] and mainly use the following lemma [5].

Lemma 3.3 (Proposition 2 in [5]). *Let $J \subset \{1, \dots, N - k - 1\}$ with $\#J = t$. Then*

$$P_J = c_{J+k+1} + \sum_{\substack{T_- \subset \{1, \dots, k+1\}, T_+ \subset J+k+1 \\ T = T_- \cup T_+, \#T=t, T_- \neq \emptyset}} c_T(\mathbf{H}_y)_{J,T}.$$

We consider the $(4, 2)$ -ISD($n, \mathbf{t}$) problem. Let the error vector be $\mathbf{e} = (\mathbf{e}_1, \dots, \mathbf{e}_4) \in \mathbb{F}_{q^m}^{4n}$, with block ranks $w_{\mathbf{e}_i} = t_i$ for $i = 1, 2, 3, 4$, and the total rank weight of the error is $t = t_1 + \dots + t_4$.

We define the following quantities for the MM attack.

Number of Variables Let $T_4(\mathbf{n}, \mathbf{t})$ denote the number of the possible non-zero maximal minors (variables) in the MaxMinors system.

To select a non-zero maximal minor, we choose t_1, t_2, t_3, t_4 columns from blocks 1, 2, 3, 4 respectively for the common rows and so

$$T_4(\mathbf{n}, \mathbf{t}) = \binom{n}{t_1} \binom{n}{t_2} \binom{n}{t_3}.$$

Number of Equations Let $N_4(n, \mathbf{t})$ denote the number of linearly independent MaxMinors equations over $\mathbb{F}_{q^m}$. The equations are indexed by subsets $J \subset \{1, \dots, 2n - 1\}$ with $|J| = t$ by Lemma 3.3.

Let $J_3 = J \cap \{1, \dots, n - 1\}$ and $J_4 = J \cap \{n, \dots, 2n - 1\}$ denote the indices corresponding to block 3 (maybe not including its first column) and block 4. For the equation to be non-zero after setting zero operation, we must have $\#J_3 \geq t_3 - 2$ and $\#J_4 \geq t_4 - 1$. Therefore,

$$N_4(n, \mathbf{t}) = \sum_{j=t_3-1}^{t_1+t_2+t_3} \binom{n-1}{j} \binom{n}{t-j}.$$

Sumarying the above discussion, we get the following result.

Proposition 3.2. *For the $(4, 2)$ -IRSD($n, \mathbf{t}$) problem over $\mathbb{F}_{q^m}$ where $t_1 \leq \dots \leq t_4$, $t = t_1 + t_2 + t_3 + t_4$, define*

$$N_4(n, \mathbf{t}) = \sum_{j=t_3-1}^{t_1+t_2+t_3} \binom{n-1}{j} \binom{n}{t-j}$$

and

$$T_4(\mathbf{n}, \mathbf{t}) = \binom{n_1}{t_1} \binom{n_2}{t_2} \binom{n_3}{t_3} \binom{n_4}{t_4},$$

where $\mathbf{n} = (n_1, n_2, n_3, n_4)$.

If $mN_4(n, \mathbf{t}) \geq T_4(\mathbf{n}, \mathbf{t}) - 1$, then the cost of MM attack to the hard problem is

$$\mathcal{O}(mN_4(n, \mathbf{t})(T_4(\mathbf{n}, \mathbf{t}))^{\omega-1}),$$

otherwise, the cost of hybrid approach is

$$\mathcal{O}\left(\min_{\substack{\mathbf{a}=(a_1, a_2, a_3, a_4) \\ mN_4(n, \mathbf{t}) \geq T_4(\mathbf{n}-\mathbf{a}, \mathbf{t})-1}} q^{\sum_{i=1}^4 a_i t_i} \cdot mN_4(n, \mathbf{t})(T_4(\mathbf{n}-\mathbf{a}, \mathbf{t}))^{\omega-1}\right).$$

3.3 Estimates on Concrete Security Strength

In this subsection we give the concrete bit complexity of the above attacks on (3, 1)-IRSD and (4, 2)-IRSD problems in Table 5, in which a and b denote the attacks complexity on the first problem and the second problem in (a, b) , respectively.

Table 5: Bit complexity of known attacks on BAG-Piglet

Instance	MM attack	PRR attack	PRR quantum	O-J attack	OJ quantum	DFR
BAG-Piglet-128	(148, 167)	(334, 210)	(184, 122)	(158, 162)	(91, 94)	2^{-130}
BAG-Piglet-256	(269, 298)	(530, 534)	(293, 295)	(395, 332)	(212, 180)	2^{-273}
BAG-Piglet-384	(397, 442)	(1379, 765)	(710, 403)	(632, 646)	(332, 338)	2^{-394}
BAG-Piglet-512	(552, 563)	(1070, 1075)	(566, 568)	(859, 763)	(446, 397)	2^{-520}

4 Performance Analysis

This section provides a reference implementation of the BAG-Piglet public-key encryption algorithm based on Augmented Gabidulin codes, including four parameter sets achieving 128, 256, 384, and 512 bits of security. The algorithm is built upon the rank-metric cryptosystem and uses arithmetic operations over the finite field $\text{GF}(2^m)$ as the basic operations. The concrete runtime performance can be found in Table 4.

To adapt to different engineering development and deployment environments, we describe the methods, techniques, and considerations used in the implementation as follows.

(1) Platform compatibility and dependency environment Language and platform selection: This implementation is written in C++, compatible with most Linux development environments, and uses a standard Makefile for compilation and management. The following libraries are required:

- NTL: Provides support for big integers and finite field arithmetic. Version 11.x or higher is recommended.
- GMP: Provides low-level big integer support and is a dependency of NTL.
- OpenSSL: Provides SHAKE hash function encapsulation, used for hash computation and randomness expansion.

Recommendation: Ensure that all dependencies are correctly installed before compilation. On common Linux distributions, these can be installed via package managers. For example, on Ubuntu, run: `sudo apt install libntl-dev libgmp-dev libssl-dev`.

(2) Programming implementation techniques

Modular design: The implementation separates functionalities such as finite field operations, Augmented Gabidulin encoding/decoding, q -polynomial processing, key and ciphertext parsing, and hash/randomness encapsulation into independent modules, which facilitates maintenance and replacement.

Finite field interface ffi: The main interface includes the following header files:

`ffi_field.h`, `ffi_elt.h`, and `ffi_vec.h`, which provide arithmetic operations over the finite field $\text{GF}(2^m)$.

The current implementation is based on NTL and can be adapted to other libraries (such as MPFQ) by replacing the underlying implementation, as long as the interface specification is followed.

Key/ciphertext representation optimization: A seed expansion approach is used to compress key sizes. The public key is represented as $(\text{seed1}, s)$, and the private key as $(\text{seed2}, x)$. Using SHAKE256, the full key structure can be reconstructed, thereby reducing communication overhead.

5 The Features of BAG-Piglet

Code-based public-key cryptography has become one of the main directions in post-quantum cryptography. In particular, rank-metric codes provide an alternative to the Hamming metric and have attracted significant attention due to their relatively compact public key sizes, which can be comparable to those of lattice-based schemes. Representative proposals include ROLLO (a merger of LAKE, LOCKER and Ouroboros-R) and RQC [11], which were selected in the second round of the NIST PQC standardization process.

However, strong algebraic attacks against rank-metric code-based schemes [4, 5] have shown that several constructions fail to achieve their claimed security levels. To maintain the desired security strength, parameter sizes must typically be increased significantly, which prevents these schemes from advancing to the third round of the NIST competition.

As a consequence, NIST has explicitly encouraged further research on rank-metric-based cryptosystems. Recently, techniques such as multiple-syndrome decoding [1, 7, 3] and block-structured noise sampling [13] have been proposed to reduce parameter sizes. The RQC-Block-MS-AG scheme demonstrates the effectiveness of integrating both approaches, achieving significant improvements [3].

Below we highlight the key features of our algorithm, followed by a comparative parameter size analysis against four typical algorithms.

5.1 The features of BAG-Piglet

The main detailed features of BAG-Piglet are summarized as follows:

Innovativeness. BAG-Piglet introduces several novel technical contributions that distinguish it from existing rank-metric-based schemes:

- It employs a multiple-syndrome technique based on Augmented Gabidulin codes, which effectively reduces parameter sizes while maintaining security.
- It provides a new effective decoding algorithm for Augmented Gabidulin codes. To the best of our knowledge, no efficient decoding algorithm for such codes was previously available; this contribution enhances both parameter efficiency and security.

- It adopts error vectors with disjoint supports as in [13], which significantly reduces the parameter size and improves the overall system performance.
- The scheme achieves a low decryption failure rate, enabling the application of the F-O transform for achieving IND-CCA2 security.

Simplicity. Compared to the original Piglet-1, BAG-Piglet simplifies the core algebraic structure by replacing the public key from a 2-dimensional matrix with a 2-dimensional vector. This design reduces structural complexity, facilitates implementation, and contributes to improved computational performance.

Flexibility and Extensibility. The block-structured design of BAG-Piglet allows fine-grained partitioning of error vectors, which provides flexibility in tuning parameters for different security levels and application scenarios. The modular use of multiple syndromes and the new decoding framework also make the scheme extensible to other rank-metric code families or future optimizations.

Compatibility. BAG-Piglet maintains compatibility with the established framework of rank-metric code-based cryptography. It builds upon standard assumptions and security models, ensuring that it can be readily integrated into existing post-quantum cryptographic infrastructures without requiring non-standard primitives or disruptive changes to system interfaces.

Performance Advantages on Variant Platforms. BAG-Piglet offers clear performance benefits over its predecessors and competing schemes:

- The number of finite field multiplications is reduced from 18 in Piglet-1 to 12, thereby effectively improving computational efficiency across both software and hardware implementations.
- In real-world communication scenarios, the public key typically does not need to be transmitted each time, whereas the ciphertext must be sent in every session. Since BAG-Piglet features a shorter ciphertext than both Piglet-1 and RQC-Block-MS-AG, it offers improved practical performance in bandwidth-constrained environments.

Comparative Parameter Size Analysis The following comparison highlights the parameter size advantages of BAG-Piglet over four typical algorithms (Piglet-1, ROLLO, RQC, and RQC-Block-MS-AG). All sizes are expressed in terms of the base parameter mn and nm , with byte-level estimates.

- **Public key size:** Approximately $2mn/82mn/8$ bytes, same as Piglet-1.
- **Ciphertext size:** Reduced from $8mn/88mn/8$ bytes in Piglet-1 to $4mn/84mn/8$ bytes in BAG-Piglet.
- **Total size (pk+ct):** Piglet-1 requires approximately $10mn/810mn/8$ bytes, while BAG-Piglet requires only $6mn/86mn/8$ bytes.

- **Advantage over RQC-Block-MS-AG:** RQC-Block-MS-AG has a public key of approximately $mn/8mn/8$ bytes and a ciphertext of approximately $4mn/84mn/8$ bytes, totalling $5mn/85mn/8$ bytes. Although BAG-Piglet has a slightly larger public key, its finer block partitioning allows a smaller parameter nn for the same security level, which results in a shorter ciphertext than RQC-Block-MS-AG. The total size (pk+ct) of BAG-Piglet is roughly comparable to that of RQC-Block-MS-AG, while offering better ciphertext efficiency for repeated communication.

5.2 Comparison with other schemes

We compare our scheme with the above four typical schemes RQC-Block-MS-AG, RQC-Block RQC, Piglet-1.

Table 6: Parameter comparison with other schemes.

Scheme	pk size	ck size	pk+ck size
RQC-Block-MS-AG-128 [3]	0.31	1.12	1.43
RQC-Block-128 [13]	0.86	1.70	2.56
RQC-128 [11]	1.83	3.65	5.5
Piglet-1-128 [14]	1.21	1.80	5.01
BAG-Piglet-128	0.58	1.10	1.68
RQC-Block-MS-AG-256 [3]	0.93	3.44	4.37
RQC-Block-256 [13]	2.42	4.83	7.25
RQC-256 [11]	4.09	5.69	9.78
Piglet-1-256 [14]	2.82	4.22	7.04
BAG-Piglet-256	1.60	3.08	4.68

References

- [1] Aguilar-Melchor, C., Aragon, N., Dyseryn, V., Gaborit, P., Zémor, G.: Lrpc codes with multiple syndromes: near ideal-size kems without ideals. In: International Conference on Post-Quantum Cryptography. pp. 45–68. Springer (2022)
- [2] Aguilar-Melchor, C., Blazy, O., Deneuville, J.C., Gaborit, P., Zémor, G.: Efficient encryption from random quasi-cyclic codes. IEEE Transactions on Information Theory **64**(5), 3927–3943 (2018)
- [3] Aragon, N., Briaud, P., Dyseryn, V., Gaborit, P., Vinçotte, A.: The blockwise rank syndrome learning problem and its applications to cryptography. In: International Conference on Post-Quantum Cryptography. pp. 75–106. Springer (2024)

- [4] Bardet, M., Briaud, P., Bros, M., Gaborit, P., Neiger, V., Ruatta, O., Tillich, J.: An algebraic attack on rank metric code-based cryptosystems. In: EUROCRYPT 2020. pp. 64–93. Springer (2020)
- [5] Bardet, M., Bros, M., Cabarcas, D., Gaborit, P., Perlner, R.A., Smith-Tone, D., Tillich, J., Verbel, J.A.: Improvements of algebraic attacks for solving the rank decoding and minrank problems. In: ASIACRYPT 2020. pp. 507–536. Springer (2020)
- [6] Berlekamp, E., McEliece, R., Van Tilborg, H.: On the inherent intractability of certain coding problems (corresp.). IEEE Transactions on Information theory **24**(3), 384–386 (1978)
- [7] Bidoux, L., Briaud, P., Bros, M., Gaborit, P.: Rqc revisited and more cryptanalysis for rank-based cryptography. IEEE Transactions on Information Theory **70**(3), 2271–2286 (2023)
- [8] Gaborit, P., Hauteville, A., Phan, D.H., Tillich, J.: Identity-based encryption from codes with rank metric. In: Katz, J., Shacham, H. (eds.) CRYPTO 2017. pp. 194–224. Springer (2017)
- [9] Gaborit, P., Zémor, G.: On the hardness of the decoding and the minimum distance problems for rank codes. IEEE Transactions on Information Theory **62**(12), 7245–7252 (2016)
- [10] Glabush, L., Hövelmanns, K., Stebila, D.: On the multi-target security of post-quantum key encapsulation mechanisms. IACR Commun. Cryptol. **3**(1), 20 (2026)
- [11] Melchor, C.A., Aragon, N., Bettaieb, S., Bidoux, L., Blazy, O., Deneuville, J.C., Gaborit, P., Zémor, G.: Rank quasi-cyclic (rqc). Second Round submission to NIST Post-Quantum Cryptography call (2020)
- [12] Puchinger, S., Renner, J., Rosenkilde, J.: Generic decoding in the sum-rank metric. IEEE Transactions on Information Theory **68**(8), 5075–5097 (2022)
- [13] Song, Y., Zhang, J., Huang, X., Wu, W.: Blockwise rank decoding problem and LRPC codes: Cryptosystems with smaller sizes. In: ASIACRYPT 2023. pp. 284–316. Springer (2023)
- [14] Wang, L.P., Hu, J.: Two new module-code-based kems with rank metric. In: Australasian Conference on Information Security and Privacy. pp. 176–191. Springer (2019)